TECHNISCHE HOCHSCHULE DEGGENDORF

Bachelor Cyber Security

**Kryptologie 2**

**Projektbericht**

# SLH-DSA / SPHINCS+ – Eine Python-Referenzimplementierung nach NIST FIPS 205


*Autoren:*

Aziz Hidri, Matrikelnummer 22410921
Mouheb Jounaidi, Matrikelnummer 22410856

*Prüfer:*

Prof. Dr. Martin Schramm


Deggendorf, den 29.07.2026

# Contents


**1 Einleitung** **2**
- 1.1 Motivation . . . 2
- 1.2 Zielsetzung . . . 2
- 1.3 Abgrenzung . . . 2
- 1.4 Eigenleistung . . . 2

**2 Systemüberblick** **2**
- 2.1 Aufbau von SLH-DSA (kurz) . . . 2
- 2.2 Technologie-Stack . . . 3

**3 Architektur** **3**
- 3.1 Modulübersicht . . . 3
- 3.2 Abhängigkeitsstruktur . . . 4
- 3.3 Datenfluss . . . 4

**4 Implementierung der Module** **4**
- 4.1 `sphincs_hilfsfunktionen.py` – Fundament . . . 4
- 4.2 `wots.py` – WOTS+ (Alg. 6–8) . . . 5
- 4.3 `xmss.py` – Merkle-Baum (Alg. 9–11) . . . 5
- 4.4 `hypertree.py` – Hypertree (Alg. 12–13) . . . 5
- 4.5 `fors.py` – FORS (Alg. 14–17) . . . 5
- 4.6 `slh_dsa.py` – Zusammenführung (Alg. 18–25) . . . 6
- 4.7 `serialisierung.py` – Byte-Layout für SK/PK/SIG . . . 6
- 4.8 `sphincs_algorithmen.py` – Kompatibilitäts-Shim . . . 6

**5 Anwendung: Streamlit-Oberfläche** **6**

**6 Automatisierte Tests** **7**

**7 Algorithmus-Zuordnung (Anhang)** **8**

**8 Bekannte Einschränkungen** **9**

**9 Fazit und Ausblick** **9**

**Quellenverzeichnis** **9**

# 1 Einleitung

## 1.1 Motivation

Mit dem Fortschritt der Quantencomputing-Forschung geraten klassische, auf Faktorisierung und diskreten Logarithmen basierende Signaturverfahren (RSA, ECDSA) mittelfristig unter Druck: Ein hinreichend leistungsfähiger Quantencomputer könnte diese Verfahren mittels Shor-Algorithmus effizient brechen. Das National Institute of Standards and Technology (NIST) hat daher im Rahmen seines Post-Quantum-Cryptography-Standardisierungsprozesses mehrere quantenresistente Verfahren standardisiert. Eines davon ist SLH-DSA (Stateless Hash-Based Digital Signature Algorithm), besser bekannt unter seinem ursprünglichen Namen SPHINCS+, standardisiert in FIPS 205. SLH-DSA stützt sich ausschließlich auf die Sicherheit kryptographischer Hashfunktionen – eine Annahme, die als besonders robust gegenüber Quantenangriffen gilt, da bislang kein effizienter Quantenangriff auf gut konstruierte Hashfunktionen bekannt ist.

## 1.2 Zielsetzung

Ziel dieses Projekts ist die vollständige, nachvollziehbare Implementierung von SLH-DSA in reinem Python, exakt nach der in FIPS 205 spezifizierten Algorithmensammlung (Algorithmen 1–25), sowie eine interaktive Streamlit-Anwendung, mit der Schlüsselerzeugung, Signieren und Verifizieren praktisch durchgeführt werden können. Das Projekt verfolgt dabei explizit *keine* Produktionsreife (siehe Kapitel 8), sondern einen didaktischen Anspruch: Jede Datei und Funktion soll möglichst direkt einem Algorithmus bzw. einer Textstelle aus FIPS 205 zuordenbar sein.

## 1.3 Abgrenzung

Die Implementierung deckt den vollständigen Signatur-Stack ab: WOTS+ (One-Time Signature), XMSS (Merkle-Baum aus WOTS+-Schlüsselpaaren), Hypertree (Verkettung mehrerer XMSS-Bäume), FORS (Few-Time-Signatur für den Nachrichten-Digest) sowie SLH-DSA selbst (Schlüsselerzeugung, „Pure"- und „PreHash"-Signaturvarianten), ergänzt um eine Serialisierungsschicht (Hex-Strings) und eine Streamlit-Weboberfläche. Nicht Gegenstand sind Nebenläufigkeits-/Performance-Optimierungen, Seitenkanalresistenz oder alternative Hash-Instanziierungen (siehe Kapitel 8).

## 1.4 Eigenleistung

Die kryptographischen Algorithmen selbst (Alg. 1–25, sowie die Datenformate aus den Figuren 15–17) sind eine möglichst wörtliche Übersetzung des in FIPS 205 vorgegebenen Pseudocodes nach Python – es wurde an dieser Stelle *kein* eigenes kryptographisches Verfahren entworfen oder inhaltlich verändert. Die eigene Leistung liegt in der Softwarearchitektur (Aufteilung des monolithischen Pseudocodes in klar abgegrenzte, einzeln testbare Module gemäß Kapitel 3), in der Serialisierungsschicht (`serialisierung.py`, technische Umsetzung der in FIPS 205 nur beschriebenen, nicht als Algorithmus formulierten Byte-Layouts) sowie insbesondere in der eigenständig entworfenen und implementierten Streamlit-Anwendung (`streamlit_sphincs.py`, Kapitel 5), die in FIPS 205 keine Entsprechung hat.

# 2 Systemüberblick

## 2.1 Aufbau von SLH-DSA (kurz)

FIPS 205 beschreibt SLH-DSA als Stapel von vier aufeinander aufbauenden Bausteinen:

1. **WOTS+** signiert einmalig einen Hash-Wert über Hash-Ketten.
2. **XMSS** fasst viele WOTS+-Schlüsselpaare in einem Merkle-Baum zusammen, sodass ein einziger öffentlicher Schlüssel (die Baumwurzel) beliebig viele WOTS+-Blätter „beglaubigt".
3. Der **Hypertree** verkettet mehrere XMSS-Bäume über mehrere Schichten, damit nicht alle

Baumblätter im Voraus berechnet werden müssen.

4. **FORS** signiert den Nachrichten-Digest mit mehreren kleinen Merkle-Bäumen parallel, um SLH-DSA gegen Mehrfachsignaturen mit demselben WOTS+-Blatt abzusichern.

### 2.2 Technologie-Stack

| Komponente | Verwendung |
|---|---|
| Python 3 | Implementierungssprache, ausschließlich Standardbibliothek |
| `hashlib.shake_256` | einzige verwendete Hash-Grundlage (SHAKE256, XOF) |
| `dataclasses` | Bündelung der SLH-DSA-Parameter (`Params`) |
| Streamlit | interaktive Weboberfläche |

Es werden bewusst keine externen Kryptographie-Bibliotheken verwendet – sämtliche Hash-Aufrufe, Baumkonstruktionen und Signaturlogik sind selbst geschrieben, um den Algorithmus vollständig transparent nachvollziehbar zu halten.

**Warum SHAKE256 statt SHA-2:** FIPS 205 erlaubt beide Hash-Familien gleichberechtigt (Abschnitt 11). SHAKE256 ist eine *Extendable-Output-Function* (XOF) und liefert nativ jede benötigte Ausgabelänge (z. B. $n \in \{16, 24, 32\}$ Byte für `F/H/T/PRF`, eine davon unabhängige Länge $m$ für `H_msg`) – ein einziger Funktionsaufruf (`hashlib.shake_256(...).digest(n)`) genügt für alle sechs Hashrollen und alle Parametersätze. Die SHA-2-Variante benötigt dagegen zusätzlich eine MGF1-Erweiterung (um aus der festen SHA-256/512-Ausgabe beliebige Längen zu erzeugen) sowie HMAC-Wrapping (SHA-2 ist als Merkle-Damgård-Konstruktion anfällig für Length-Extension-Angriffe, SHAKE256 ist als Sponge-Konstruktion von Natur aus dagegen immun). SHAKE256 wurde also nicht aus Sicherheitsgründen gewählt – FIPS 205 stuft beide Varianten als gleichwertig sicher ein –, sondern weil es für diese Implementierung deutlich weniger zusätzlichen Code (keine MGF1-/HMAC-Hilfsfunktionen) bei identischem didaktischen Nutzen bedeutet.

## 3 Architektur

### 3.1 Modulübersicht

Der Code bildet den in FIPS 205 beschriebenen Schichtenstapel als Modul-Hierarchie ab: Jede Datei entspricht genau einer Schicht des Standards, ergänzt um zwei Querschnittsmodule (Hilfsfunktionen, Serialisierung) und zwei äußere Schichten (SLH-DSA-Schnittstelle, Weboberfläche).

| Datei | Verantwortlichkeit | Alg. |
|---|---|---|
| `sphincs_hilfsfunktionen.py` | Bit-/Byte-Konvertierung, ADRS, Hashfunktionen, Parameter | 1–5 |
| `wots.py` | Einmal-Signatur WOTS+ | 6–8 |
| `xmss.py` | Merkle-Baum aus WOTS+-Schlüsselpaaren | 9–11 |
| `hypertree.py` | Verkettung mehrerer XMSS-Bäume | 12–13 |
| `fors.py` | Few-Time-Signatur für Nachrichten-Digest | 14–17 |
| `slh_dsa.py` | Zusammenführung zur vollständigen Schnittstelle | 18–25 |
| `serialisierung.py` | Byte-/Hex-Kodierung von SK, PK, SIG | – |
| `sphincs_algorithmen.py` | Kompatibilitäts-Shim (Re-Export) | – |
| `streamlit_sphincs.py` | interaktive Anwendung (Einstiegspunkt) | – |

### 3.2 Abhängigkeitsstruktur

Die Abhängigkeitsrichtung ist strikt einseitig (keine Zirkelbezüge) und entspricht der Konstruktionsreihenfolge von SPHINCS+ selbst:

```
streamlit_sphincs.py
       |
       v
    slh_dsa.py -----------------+
    / \ |
   v v v
hypertree.py fors.py xmss.py
   | ^
   v |
xmss.py ------------------------+ (direkter Import fuer xmss_node
   | in slh_keygen_internal)
   v
wots.py
   |
   v
sphincs_hilfsfunktionen.py

serialisierung.py (unabhaengig, nur Hex-Ein-/Ausgabe)
sphincs_algorithmen.py (nur Re-Export, Altlast-Kompatibilitaet)
```

Jede Datei importiert ausschließlich aus Modulen einer „tieferen“ Schicht. `fors.py` importiert dabei ausschließlich aus `sphincs_hilfsfunktionen.py` und ist damit – anders als `hypertree.py` – vollständig unabhängig von `xmss.py`. `slh_dsa.py` importiert `xmss_node` zusätzlich direkt aus `xmss.py` (für die Schlüsselerzeugung), unabhängig von `hypertree.py`. Dadurch ist z. B. `wots.py` vollständig unabhängig von `xmss.py`, `fors.py` oder `slh_dsa.py` testbar, während `slh_dsa.py` alle darunterliegenden Bausteine kombiniert.

### 3.3 Datenfluss

Schlüsselerzeugung ruft rekursiv einen XMSS-Baum bis zur Wurzel auf: `slh_keygen(P)` → `slh_keygen_internal(...)` → `xmss_node(SK_seed, 0, hp, PK_seed, ADRS, P)` (rekursiv) → `wots_pkGen(...)` an jedem Blatt.

Signieren durchläuft FORS und anschließend den Hypertree: aus $R =$ `PRF_msg`$(\dots)$ und `digest` $=$ `H_msg`$(R, \texttt{PK_seed}, \texttt{PK_root}, M)$ werden `md`, `idx_tree`, `idx_leaf` extrahiert; `fors_sign` signiert `md`, `fors_pkFromSig` rekonstruiert `PK_FORS`, und `ht_sign` signiert `PK_FORS` Schicht für Schicht (0 bis $d-1$).

Verifizieren spiegelt denselben Ablauf ohne Zugriff auf `SK_seed`: Das durchgängige Prinzip lautet, dass Verifikation nie eine separate Prüf-Logik aufruft, sondern aus der Signatur denselben Wert rekonstruiert, den auch die Schlüsselerzeugung berechnet hätte (`wots_pkFromSig`, `xmss_pkFromSig`, `fors_pkFromSig` sind die Rekonstruktions-Gegenstücke zu `wots_pkGen`, `xmss_node`, `fors_node`) und vergleicht das Ergebnis mit `PK_root`.

## 4 Implementierung der Module

### 4.1 `sphincs_hilfsfunktionen.py` – Fundament

Bündelt alle Bausteine, die von mehreren höheren Schichten benötigt werden:

- **Alg. 1–4** (`gen_len2`, `toInt`, `toByte`, `base_2b`): Bit-/Byte-Konvertierung, u. a. für die Checksummenkodierung in WOTS+ und die Indexberechnung in FORS.
- **ADRS-Klasse**: bildet die 32-Byte-Adressstruktur (Abschnitt 4.2/4.3) als Objekt mit sechs Feldern (`layer`, `tree`, `type`, `w4`, `w5`, `w6`) ab; Setter wie `setChainAddress` kapseln, dass dieselben Rohfelder je nach Adresstyp unterschiedlich interpretiert werden.

- **Hashfunktionen** `PRF_msg`, `H_msg`, `PRF`, `F`, `H`, `T`: durchgängig als eine einzige SHAKE256-Instanziierung implementiert (`hashlib.shake_256(...).digest(n)`).
- **chain** (Alg. 5): die WOTS+-Hash-Kette – iteriert `F` $s$-mal ab Startindex $i$ unter laufender Aktualisierung der Kettenadresse in `ADRS`.
- **Params-Dataclass**: bündelt die Kernparameter ($n$, `lgw`, $h'$, $d$, $k$, $a$) und berechnet abgeleitete Größen ($w$, `len1`, `len2`, `len_`, $h$, $m$) automatisch in `__post_init__`.
- **NIST_PARAMETERSAETZE**: die sechs offiziellen Parametersätze aus FIPS 205, Tabelle 2 (128s/128f/192s/192f/256s/256f).

```
@dataclass
class Params:
    n: int; lgw: int; hp: int; d: int; k: int; a: int
    w: int = field(init=False); len1: int = field(init=False)
    len2: int = field(init=False); len_: int = field(init=False)
    h: int = field(init=False); m: int = field(init=False)

    def __post_init__(self):
        self.w = 2 ** self.lgw
        self.len1 = -(-(8 * self.n) // self.lgw)
        self.len2 = gen_len2(self.n, self.lgw)
        self.len_ = self.len1 + self.len2
        self.h = self.hp * self.d
        self.m = (-(-(self.h - self.hp) // 8) + -(-self.hp // 8)
                  + -(-(self.k * self.a) // 8))
```

### 4.2 `wots.py` – WOTS+ (Alg. 6–8)

`wots_pkGen` erzeugt für jede der `len_` Hash-Ketten einen Geheimwert per PRF, läuft die Kette vollständig bis $w-1$ und komprimiert alle Kettenenden mit `T` zu einem Public Key. Die interne Hilfsfunktion `_msg_und_checksumme` (kein eigener FIPS-205-Algorithmus, in Alg. 7/8 identisch und daher dedupliziert) berechnet die Basis-$w$-Zerlegung der Nachricht sowie die Checksumme. `wots_sign` führt jede Kette nur bis zum kodierten Index aus; `wots_pkFromSig` rekonstruiert aus einer Signatur den zugehörigen Public Key, indem die Kette bis zum Ende weitergeführt wird – das Grundprinzip, das Verifikation ohne separate Prüf-Logik ermöglicht.

### 4.3 `xmss.py` – Merkle-Baum (Alg. 9–11)

`xmss_node` ist rekursiv implementiert: Blattknoten ($z = 0$) sind WOTS+-Public-Keys, innere Knoten ($z > 0$) ergeben sich als $H(\text{links} \,\|\, \text{rechts})$. `xmss_sign` berechnet den Authentifizierungspfad über $h'$ Aufrufe von `xmss_node` sowie die WOTS+-Signatur des betroffenen Blatts. `xmss_pkFromSig` rekonstruiert die Baumwurzel analog zum WOTS+-Rekonstruktionsprinzip.

### 4.4 `hypertree.py` – Hypertree (Alg. 12–13)

Verkettet $d$ XMSS-Bäume, damit ein einziger Top-Level-Schlüssel (`PK.root`) eine große Zahl an XMSS-Bäumen autorisieren kann, ohne dass diese vollständig im Voraus berechnet werden müssen. `ht_sign` signiert zunächst mit dem untersten XMSS-Baum (Schicht 0); dessen Wurzel wird von der jeweils nächsthöheren Schicht signiert, bis Schicht $d-1$ erreicht ist – `idx_tree`/`idx_leaf` werden dabei schichtweise per Bit-Shift ($\gg h'$) heruntergebrochen. `ht_verify` durchläuft denselben Pfad und vergleicht das Endergebnis mit `PK_root`.

### 4.5 `fors.py` – FORS (Alg. 14–17)

FORS signiert den Nachrichten-Digest mit $k$ unabhängigen, kleinen Merkle-Bäumen der Höhe $a$ statt mit einem einzelnen WOTS+-Blatt. `fors_skGen` erzeugt PRF-basierte Geheimwerte analog zu WOTS+; `fors_node` ist rekursiv wie `xmss_node`, jedoch für $k$ unabhängige Bäume. `fors_sign` wählt für jeden der $k$ Bäume anhand der Nachricht (`base_2b(md, a, k)`) einen In-

dex aus und liefert den zugehörigen Geheimwert samt Authentifizierungspfad. `fors_pkFromSig` rekonstruiert alle $k$ Baumwurzeln und komprimiert sie mit `T` zu einem einzigen FORS-Public-Key, der anschließend in `slh_dsa.py` wie eine „Nachricht“ vom Hypertree signiert bzw. verifiziert wird.

### 4.6 `slh_dsa.py` – Zusammenführung (Alg. 18–25)

`slh_keygen_internal`/`slh_keygen` erzeugen `SK_seed`, `SK_prf`, `PK_seed` zufällig (`os.urandom`) und berechnen `PK_root` als Wurzel des obersten XMSS-Baumes. Die interne Hilfsfunktion `_digest_aufteilen` (gemeinsamer Kern aus Alg. 19/20) spaltet den `H_msg`-Digest in `md`, `idx_tree`, `idx_leaf` auf. `slh_sign` (22)/`slh_verify` (24) sind die „Pure“-Variante – sie betten nur den Kontextstring `ctx` in die Nachricht ein ($\texttt{0x00} \,\|\, \text{len}(\texttt{ctx}) \,\|\, \texttt{ctx} \,\|\, M$). `hash_slh_sign` (23)/`hash_slh_verify` (25) sind die „PreHash“-Variante für lange Nachrichten – sie hashen $M$ vorab extern (wählbar zwischen SHA-256, SHA-512, SHAKE128, SHAKE256) und betten stattdessen $\texttt{0x01} \,\|\, \text{len}(\texttt{ctx}) \,\|\, \texttt{ctx} \,\|\, \text{OID(PH)} \,\|\, \text{PH}$ ein.

### 4.7 `serialisierung.py` – Byte-Layout für SK/PK/SIG

Technische Umsetzung der in FIPS 205 beschriebenen, aber nicht als Pseudocode formulierten Datenformate (Figuren 15–17):

| Objekt | Byte-Layout | Länge |
|---|---|---|
| SK (Fig. 15) | `SK.seed`‖`SK.prf`‖`PK.seed`‖`PK.root` | $4n$ |
| PK (Fig. 16) | `PK.seed`‖`PK.root` | $2n$ |
| SIG (Fig. 17) | $R$‖`SIG_FORS`‖`SIG_HT` | $n(1{+}k(1{+}a){+}h{+}d{\cdot}\texttt{len_})$ |

`sig_from_bytes` berechnet die erwartete Länge vorab aus den aktuell gewählten Parametern und erkennt so sofort, ob ein eingegebener Hex-String zu den gewählten Parametern passt. Diese Schicht existiert, weil SK/PK/SIG intern als Python-Tupel aus `bytes` repräsentiert werden, für Wiederverwendung in einer anderen Sitzung jedoch als zusammenhängender Hex-String vorliegen müssen – dies spiegelt wider, dass Signieren (Alg. 22) nur $M$, `ctx`, SK und Verifizieren (Alg. 24) nur $M$, SIG, `ctx`, PK benötigen: beide Operationen sind bewusst vollständig unabhängig voneinander.

### 4.8 `sphincs_algorithmen.py` – Kompatibilitäts-Shim

Enthält keine eigene Logik, sondern importiert alle Namen aus den obigen Modulen unter einem gemeinsamen, historisch gewachsenen Namensraum. Dient ausschließlich der Abwärtskompatibilität; neuer Code importiert direkt aus den spezialisierten Modulen.

## 5 Anwendung: Streamlit-Oberfläche

`streamlit_sphincs.py` ist der einzige Einstiegspunkt mit Benutzerinteraktion (`streamlit run streamlit_sphincs.py`) und läuft in fünf Schritten ab:

1. **Parameterwahl** – vordefinierter NIST-Parametersatz oder frei wählbare Werte für $n$, $h'$, $d$, $k$, $a$, `lgw`.
2. **Operationswahl** – Schlüsselerzeugung, Signieren oder Verifizieren; jede Operation blendet nur die dafür nötigen Eingabefelder ein.
3. **Kostenschätzung** – die Heuristik $\texttt{kosten} = d \cdot 2^{h'} + k \cdot 2^{a}$ schätzt den Rechenaufwand ab; oberhalb eines Schwellwerts wird gewarnt, oberhalb eines höheren die Berechnung abgelehnt.
4. **Ausführung** – je nach Operation wird `slh_keygen`, `slh_sign`/`hash_slh_sign` bzw. `slh_verify`/`hash_slh_verify` aufgerufen; Ergebnisse werden über `serialisierung.py` in Hex kodiert.
5. **Ausgabe** – Kurzdarstellung sowie vollständiger Hex-String zum Kopieren; das Verifika-

tionsergebnis wird als „gültig“/„ungültig“ dargestellt, jeweils zusammen mit der gemessenen Rechenzeit der Operation (`time.perf_counter()`).

Weicht bei einem gewählten NIST-Preset einer der Werte $n$, $h'$, $d$, $k$, $a$ nachträglich vom offiziellen Wert ab (die Eingabefelder bleiben bewusst editierbar, siehe Schritt 1), zeigt die Oberfläche eine Warnung: die für diesen Parametersatz zugesicherte Sicherheitsstufe gilt dann nicht mehr, auch wenn der Preset-Name im Dropdown unverändert angezeigt wird.

Für kleine, selbst gewählte Parameter (Standardwerte: $n$=4, $h'$=3, $d$=2, $k$=4, $a$=3) laufen alle drei Operationen in Sekundenbruchteilen. Alle sechs offiziellen NIST-Parametersätze wurden zusätzlich vollständig getestet (Schlüsselerzeugung, Signieren, Verifizieren – jeweils mit anschließender erfolgreicher Verifikation); die folgende Tabelle zeigt die gemessenen Laufzeiten auf der Entwicklungsumgebung:

| **Parametersatz** | **Keygen** | **Signieren** | **Verifizieren** |
|---|---|---|---|
| SLH-DSA-SHAKE-128s | 1,12 s | 8,50 s | 0,01 s |
| SLH-DSA-SHAKE-128f | 0,02 s | 0,42 s | 0,03 s |
| SLH-DSA-SHAKE-192s | 1,55 s | 14,53 s | 0,01 s |
| SLH-DSA-SHAKE-192f | 0,02 s | 0,65 s | 0,03 s |
| SLH-DSA-SHAKE-256s | 1,01 s | 13,08 s | 0,02 s |
| SLH-DSA-SHAKE-256f | 0,07 s | 1,33 s | 0,04 s |

Entgegen der ursprünglichen Annahme, die vollen NIST-Parametersätze seien für diese Implementierung nur als Referenz ohne praktischen Nutzen hinterlegt, zeigt sich: alle sechs Parametersätze sind auch live in der Streamlit-Oberfläche nutzbar – die „f“-Varianten in Sekundenbruchteilen, die „s“-Varianten mit einer Signierzeit von bis zu ca. 15 Sekunden, was für eine interaktive Demo (mit Ladeanzeige und Kostenwarnung) vertretbar ist. Die in Kapitel 8/9 genannte fehlende Performance-Optimierung betrifft also nicht die grundsätzliche Nutzbarkeit, sondern nur den Komfort bei sehr häufiger, wiederholter Nutzung.

# 6 Automatisierte Tests

Zusätzlich zur manuellen Prüfung über die Streamlit-Oberfläche existiert die Datei `test_slh_dsa.py`, die die Korrektheit der Implementierung automatisiert und wiederholbar nachweist. Sie verwendet ausschließlich das in Python bereits enthaltene `unittest`-Modul (keine zusätzliche Bibliothek nötig) und wird mit

```
python -m unittest test_slh_dsa.py -v
```

ausgeführt. Ein `unittest` ist ein automatisiertes Testskript: Statt die App per Hand zu bedienen und das Ergebnis selbst zu beurteilen, ruft der Code direkt die Funktionen auf (`slh_keygen`, `slh_sign`, `slh_verify`, ...) und vergleicht das Ergebnis automatisch mit dem erwarteten Wert (`assertTrue`, `assertEqual` usw.). Schlägt eine dieser Prüfungen fehl, meldet `unittest` genau, welcher Test und welche Zeile betroffen ist – das erlaubt es, nach jeder Änderung am Code sofort und ohne manuelles Nachprüfen zu sehen, ob die Implementierung noch korrekt arbeitet.

Die Testdatei ist in zwei Klassen gegliedert:

- `TestNistParametersaetze` – der eigentliche Korrektheitsnachweis. Für jeden der sechs offiziellen NIST-Parametersätze aus Tabelle 2 in FIPS 205 (nicht mit frei erfundenen Werten) läuft ein vollständiger Rundlauf: Schlüsselerzeugung, Signieren und erfolgreiches Verifizieren sowohl in der „Pure“- als auch der „HashSLH-DSA“-Variante, ein Rundlauf durch die Hex-Serialisierung (`serialisierung.py`), sowie die Kontrolle, dass eine nachträglich veränderte Nachricht zuverlässig als ungültig erkannt wird. Jeder Parametersatz besitzt eine eigene, nach ihm benannte Testmethode (z. B. `test_slh_dsa_shake_256s`), damit in der Ausgabe sofort erkennbar ist, welcher Satz betroffen wäre, falls ein Test fehlschlägt.

- **`TestRandfaelle`** – schnelle Zusatztests mit klein gewählten, selbstdefinierten Parametern für Verhaltensdetails, die nichts mit der Sicherheitsstufe zu tun haben: alle vier wählbaren Pre-Hash-Funktionen (SHA-256, SHA-512, SHAKE128, SHAKE256), eine leere Nachricht, ein zu langer Kontextstring (`ctx` $>$ 255 Byte, muss laut FIPS 205 abgelehnt werden) sowie die Ablehnung eines falschen Public Keys.

Der vollständige Testlauf (10 Tests, alle sechs NIST-Parametersätze plus vier Randfälle) wurde erfolgreich ausgeführt:

```
test_slh_dsa_shake_128f ... ok
test_slh_dsa_shake_128s ... ok
test_slh_dsa_shake_192f ... ok
test_slh_dsa_shake_192s ... ok
test_slh_dsa_shake_256f ... ok
test_slh_dsa_shake_256s ... ok
test_falscher_public_key_wird_abgelehnt ... ok
test_kontextstring_zu_lang_wird_abgelehnt ... ok
test_leere_nachricht ... ok
test_verschiedene_pre_hash_funktionen ... ok

----------------------------------------------------------------------
Ran 10 tests in 93.746s

OK
```

Die Laufzeit von rund 1,5 Minuten resultiert hauptsächlich aus den „s“-Varianten (größere Baumhöhe $h'$, siehe Benchmark-Tabelle in Kapitel 5) und bestätigt, dass die Implementierung nicht nur mit Demo-Parametern, sondern mit allen tatsächlich zugesicherten Sicherheitsstufen korrekt und reproduzierbar funktioniert.

# 7 Algorithmus-Zuordnung (Anhang)

| **FIPS-205-Algorithmus** | **Funktion** | **Datei** |
|---|---|---|
| 1–5 (`gen_len2`, `toInt`, `toByte`, `base_2b`, `chain`) | gleichnamige Funktionen | `sphincs_hilfsfunktionen.py` |
| 6 – `wots_pkGen` | `wots_pkGen` | `wots.py` |
| 7 – `wots_sign` | `wots_sign` | `wots.py` |
| 8 – `wots_pkFromSig` | `wots_pkFromSig` | `wots.py` |
| 9 – `xmss_node` | `xmss_node` | `xmss.py` |
| 10 – `xmss_sign` | `xmss_sign` | `xmss.py` |
| 11 – `xmss_pkFromSig` | `xmss_pkFromSig` | `xmss.py` |
| 12 – `ht_sign` | `ht_sign` | `hypertree.py` |
| 13 – `ht_verify` | `ht_verify` | `hypertree.py` |
| 14 – `fors_skGen` | `fors_skGen` | `fors.py` |
| 15 – `fors_node` | `fors_node` | `fors.py` |
| 16 – `fors_sign` | `fors_sign` | `fors.py` |
| 17 – `fors_pkFromSig` | `fors_pkFromSig` | `fors.py` |
| 18 – `slh_keygen_internal` | `slh_keygen_internal` | `slh_dsa.py` |
| 19 – `slh_sign_internal` | `slh_sign_internal` | `slh_dsa.py` |
| 20 – `slh_verify_internal` | `slh_verify_internal` | `slh_dsa.py` |
| 21 – `slh_keygen` | `slh_keygen` | `slh_dsa.py` |
| 22 – `slh_sign` | `slh_sign` | `slh_dsa.py` |
| 23 – `hash_slh_sign` | `hash_slh_sign` | `slh_dsa.py` |
| 24 – `slh_verify` | `slh_verify` | `slh_dsa.py` |
| 25 – `hash_slh_verify` | `hash_slh_verify` | `slh_dsa.py` |
| Figuren 15–17 (kein Algorithmus) | `*_to_bytes`/`*_from_bytes`/`*_hex` | `serialisierung.py` |

# 8 Bekannte Einschränkungen

- **Nicht constant-time / nicht seitenkanalresistent.** Die Implementierung dient Demonstrations- und Lernzwecken, nicht dem produktiven Einsatz mit echten Geheimnissen.
- **Einheitliche Hash-Instanziierung.** Alle sechs Hash-Funktionsrollen (`PRF_msg`, `H_msg`, `PRF`, `F`, `H`, `T`) verwenden durchgängig SHAKE256 statt der in FIPS 205 ebenfalls vorgesehenen SHA-2-Alternative.
- **Kein Caching zwischen Baumknoten.** `xmss_node` und `fors_node` berechnen bei jedem Aufruf den vollständigen Teilbaum neu; der Aufwand wächst exponentiell mit der Baumhöhe – für Demo-Parameter unproblematisch, für volle NIST-Parametersätze der Grund für die Laufzeitwarnung in der Oberfläche.
- **Zufallszahlenerzeugung.** `os.urandom` wird direkt als Zufallsquelle verwendet, ohne den in FIPS 205 optional vorgesehenen zusätzlichen DRBG-Wrapper.

# 9 Fazit und Ausblick

Das Projekt zeigt, dass sich der vollständige SLH-DSA-Algorithmenstapel (Alg. 1–25 aus FIPS 205) in reinem Python in eine klar geschichtete, nachvollziehbare Modulstruktur überführen lässt, die sich direkt an der Struktur des Standards orientiert. Durch die konsequente Trennung von Hilfsfunktionen, WOTS+, XMSS, Hypertree, FORS und der SLH-DSA-Zusammenführung ist jede Komponente einzeln nachvollziehbar und mit der zugehörigen Textstelle in FIPS 205 abgleichbar. Die Streamlit-Oberfläche macht die sonst abstrakten Objekte (Schlüssel, Signaturen) greifbar und erlaubt es, den Effekt unterschiedlicher Parametersätze auf Signaturgröße und Laufzeit direkt zu beobachten.

Mögliche Erweiterungen für zukünftige Arbeiten:

- Performance-Optimierung (Parallelisierung, Caching bereits berechneter Teilbäume): alle 6 NIST-Parametersätze wurden bereits erfolgreich getestet (Tabelle in Kapitel 5, bis ca. 15 s pro Signatur bei den „s“-Varianten) – eine Optimierung würde vor allem den Komfort bei häufiger, wiederholter Nutzung verbessern, ist für die grundsätzliche Funktionsfähigkeit aber nicht mehr nötig.
- Ergänzung einer zweiten Hash-Instanziierung (SHA-2-basiert) zum direkten Vergleich mit der aktuell ausschließlich verwendeten SHAKE256-Variante.
- Validierung gegen die offiziellen NIST-Testvektoren (feste Eingabe-/Ausgabe-Byte-Paare), zusätzlich zur bereits vorhandenen automatisierten Testsuite (Kapitel 6), um die Implementierung auch gegen eine externe Referenz statt nur gegen sich selbst zu validieren.

# Quellenverzeichnis

National Institute of Standards and Technology (NIST): *FIPS 205 – Stateless Hash-Based Digital Signature Standard*, August 2024.